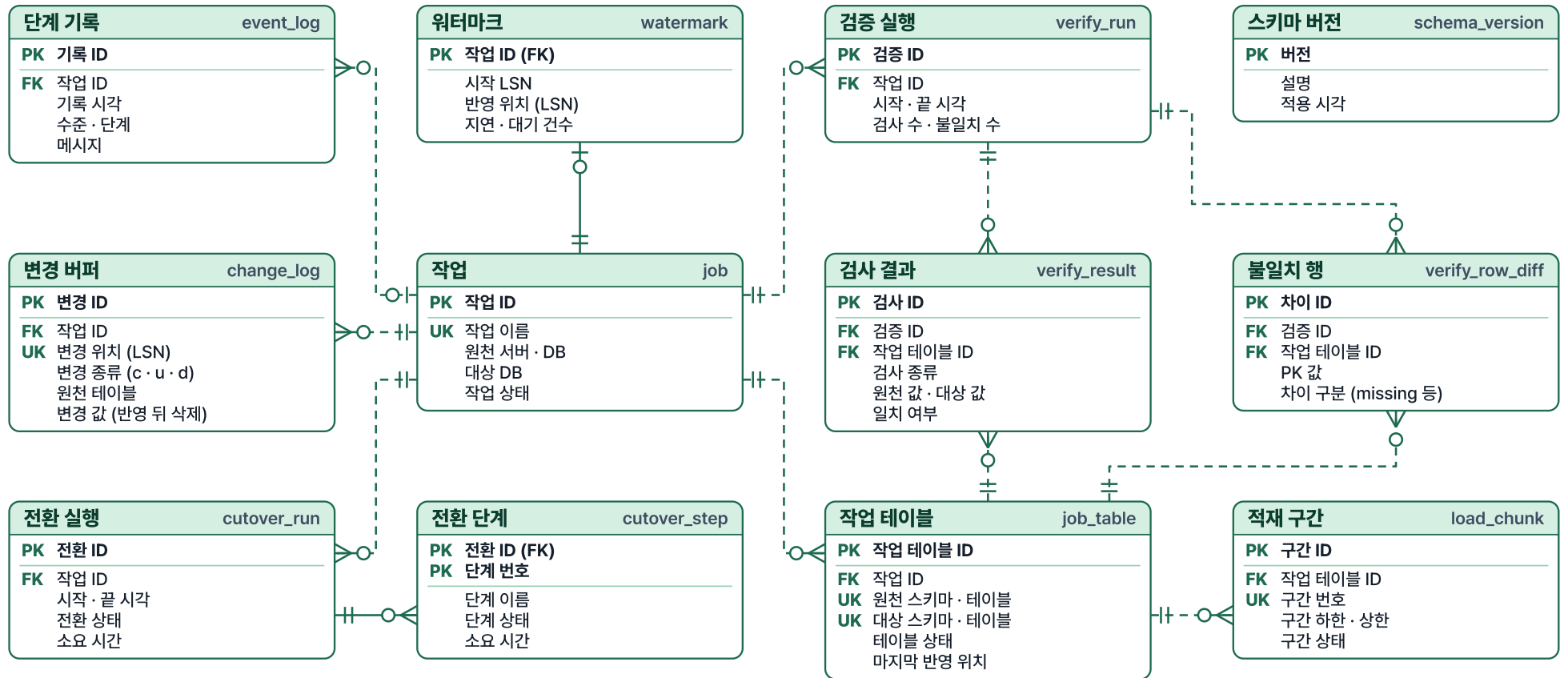


# KDMS 관리 테이블 논리 ERD (1/3)

엔터티-관계 핵심 속성 · 머리 오른쪽은 물리 테이블 이름 · 컬럼-타입 전체는 2-3장 물리 ERD

KDMS-D05 · v0.3 · 2026-10-09 · 1/3



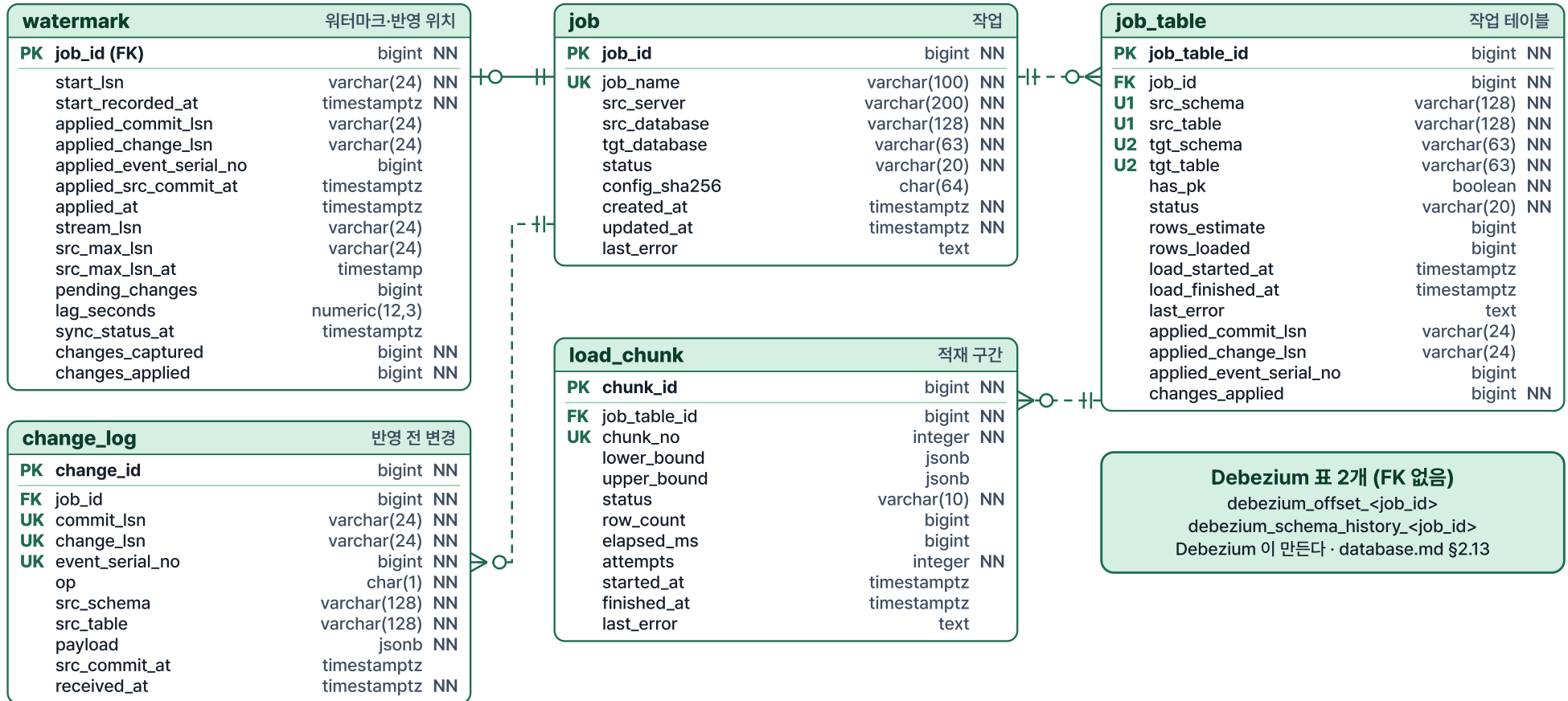
범례 —|| 정확히 1 —○+ 0 또는 1 —○< 0 이상 (N) — 식별 관계 - - - 비식별 관계

작업 1건 = 원천 DB 1 → 대상 DB 1 · 스키마 버전은 관계 없음  
 행 값은 변경 버퍼에만, 반영하면 바로 지운다

# KDMS 관리 테이블 물리 ERD (2/3) 작업·적재·동기화

대상 PostgreSQL 의 kdms 스키마 · 관리 스키마 v3 · IE 표기 · 컬럼 정의는 docs/database.md

KDMS-D05 · v0.3 · 2026-10-09 · 2/3



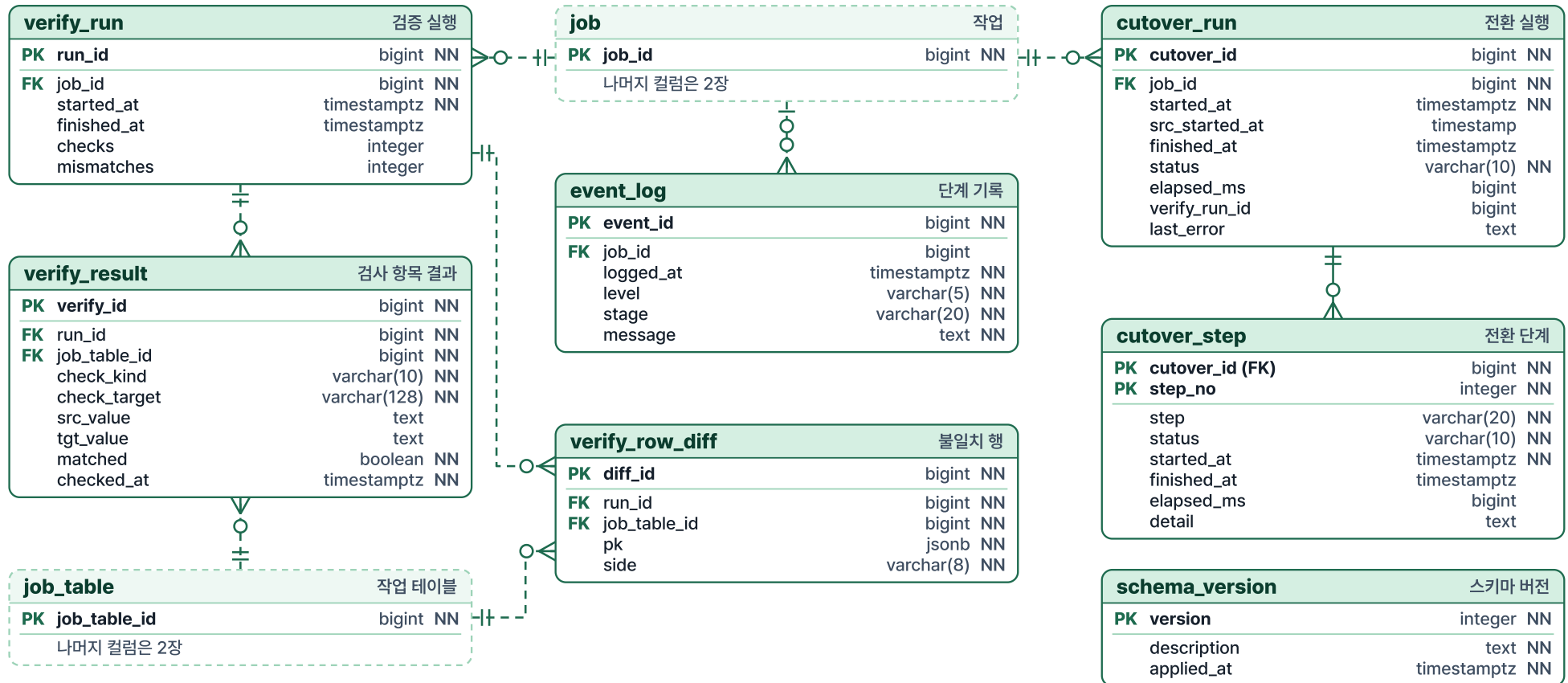
범례 —|| 정확히 1 —○+ 0 또는 1 —○< 0 이상 (N) — 식별 관계 - - - 비식별 관계

U1 · U2 = job\_id 를 포함한 복합 UK · UK = FK 컬럼을 포함한 복합 UK  
 FK 는 모두 ON DELETE CASCADE · NN = NOT NULL

# KDMS 관리 테이블 물리 ERD (3/3) 검증·전환·기록

점선 테두리 = 2장 엔터티를 참조로 다시 그림(PK 만) · 컬럼 정의는 docs/database.md

KDMS-D05 · v0.3 · 2026-10-09 · 3/3



범례 —|| 정확히 1 —○+ 0 또는 1 —○< 0 이상 (N) — 식별 관계 - - - 비식별 관계

event\_log.job\_id 는 NULL 허용 · verify\_run\_id 는 FK 아님  
FK 는 모두 ON DELETE CASCADE · NN = NOT NULL

## 1. 읽는 법

- 1장은 논리 ERD 다. 엔터티를 한글 이름으로, 관계와 핵심 속성(식별자·외래 식별자·주요 속성)만 그렸다. 머리 오른쪽 글은 그 엔터티를 구현한 물리 테이블 이름이다.
- 2·3장은 물리 ERD 다. 대상 PostgreSQL 의 `kdms` 스키마에 실제로 있는 테이블·컬럼·타입을 그대로 그렸다. 2장 작업·적재·동기화, 3장 검증·전환·기록.
- IE(Information Engineering, 정보 공학) 표기. 선 끝 기호로 관계 차수를 읽는다: 두 줄 = 정확히 1, 원 + 한 줄 = 0 또는 1, 원 + 까마귀발 = 0 이상(N). 부모 쪽이 1, 자식(FK 를 가진 쪽)이 N 이다.
- 실선 = 식별 관계(부모 PK 가 자식 PK 에 들어감: `watermark`, `cutover_step`), 점선 = 비식별 관계.
- 물리 엔터티 박스: 머리 = 테이블 이름과 한글 이름, 구분선 위 = PK, 아래 = 나머지 컬럼. 왼쪽 표시 PK·FK·UK, 오른쪽 NN = NOT NULL(빈칸은 NULL 허용). `((FK))` 는 PK 이면서 FK 인 컬럼.
- 3장의 점선 테두리 엔터티(`job`, `job_table`)는 2장 엔터티를 관계를 보이려고 다시 그린 것이다.
- 컬럼 설명·기본값·CHECK·인덱스 전체는 `docs/database.md` §2 한 곳에 둔다. 이 그림은 그 문서와 같은 순서로 컬럼을 그렸다.

## 2. 카·인덱스

| 테이블                     | PK                      | UK                                                                     | FK (모두 ON DELETE CASCADE)  | 인덱스                                         |
|-------------------------|-------------------------|------------------------------------------------------------------------|----------------------------|---------------------------------------------|
| <code>job</code>        | job_id (identity)       | job_name                                                               | —                          | —                                           |
| <code>job_table</code>  | job_table_id (identity) | U1 (job_id, src_schema, src_table), U2 (job_id, tgt_schema, tgt_table) | job_id → job               | ix_job_table_job (job_id)                   |
| <code>load_chunk</code> | chunk_id (identity)     | (job_table_id, chunk_no)                                               | job_table_id → job_table   | ix_load_chunk_status (job_table_id, status) |
| <code>watermark</code>  | job_id                  | —                                                                      | job_id → job (식별, 1: 0..1) | —                                           |

| 테이블                                                 | PK                     | UK                                                                  | FK (모두 ON DELETE CASCADE)                     | 인덱스                                                                                          |
|-----------------------------------------------------|------------------------|---------------------------------------------------------------------|-----------------------------------------------|----------------------------------------------------------------------------------------------|
| <code>change_log</code>                             | change_id (identity)   | uq_change_log_pos (job_id, commit_lsn, change_lsn, event_serial_no) | job_id → job                                  | ix_change_log_apply (job_id, src_schema, src_table, commit_lsn, change_lsn, event_serial_no) |
| <code>verify_run</code>                             | run_id (identity)      | —                                                                   | job_id → job                                  | —                                                                                            |
| <code>verify_result</code>                          | verify_id (identity)   | —                                                                   | run_id → verify_run, job_table_id → job_table | ix_verify_result_run (run_id)                                                                |
| <code>verify_row_diff</code>                        | diff_id (identity)     | —                                                                   | run_id → verify_run, job_table_id → job_table | ix_verify_row_diff_run (run_id, job_table_id)                                                |
| <code>event_log</code>                              | event_id (identity)    | —                                                                   | job_id → job (NULL 허용, 0..1 : N)              | ix_event_log_job (job_id, logged_at)                                                         |
| <code>cutover_run</code>                            | cutover_id (identity)  | —                                                                   | job_id → job                                  | ix_cutover_run_job (job_id, cutover_id)                                                      |
| <code>cutover_step</code>                           | (cutover_id, step_no)  | —                                                                   | cutover_id → cutover_run (식별)                 | —                                                                                            |
| <code>schema_version</code>                         | version                | —                                                                   | —                                             | —                                                                                            |
| <code>debezium_offset_&lt;job_id&gt;</code>         | 없음                     | —                                                                   | —                                             | Debezium 이 만든다. 저장할 때마다 표 전체를 다시 쓴다                                                          |
| <code>debezium_schema_history_&lt;job_id&gt;</code> | (id, history_data_seq) | —                                                                   | —                                             | Debezium 이 만든다                                                                               |

`cutover_run.verify_run_id` 는 이 전환에서 돌린 검증을 가리키지만 FK 가 아니다. 버전별 추가 컬럼(v2: `job_table.applied_*`·`changes_applied`, `watermark` 9~16번, v3: `cutover_*`)은 `docs/database.md` §2·§4.

### 3. 상태 값 (CHECK)

- `job.status`: `SCHEMA_DONE → LOADING → SYNCING → CUTOVER → VERIFIED → DONE`, 전환 단계가 실패하면 `FAILED` (다시 load-sync-cutover 가능). `PLANNED` 는 기본값이지만 지금 코드는 쓰지 않는다. `kdms reset --yes` 는 `SCHEMA_DONE` 으로 되돌린다.
- `job_table.status`: `PENDING → LOADING → LOADED`, 구간이 실패하면 `FAILED` 뒤 다시 적재. 반영기는 `LOADED` 테이블의 변경만 적용한다.
- `load_chunk.status`: `PENDING → RUNNING → DONE | FAILED`. 다시 적재하면 `DONE` 이 아닌 구간만 처음부터.
- `cutover_run.status`: `RUNNING · DONE · FAILED`, `cutover_step.status`: 여기에 `SKIPPED`. `change_log.op`: `c · u · d`, `verify_result.check_kind`: `count · sum · hash`, `verify_row_diff.side`: `missing · extra · diff`, `event_log.level`: `INFO · WARN · ERROR`.
- 상태 전이 그림과 명령별 허용 상태는 `docs/database.md` §3.

### 4. 원칙

- 상태는 대상 PG 한 곳(`kdms` 스키마)에 둔다. 별도 파일·메시지 브로커가 없다.
- 행 값(개인정보)·비밀번호를 남기는 컬럼은 없다. 예외는 `change_log.payload` 이며 반영하면 바로 지운다. `verify_row_diff.pk` 는 PK 값만, 테이블마다 앞 100건.
- 정의를 바꾸면 새 버전 SQL 파일을 더하고, 앱이 시작할 때 차례로 적용한다(v1 1단계, v2 4단계, v3 5단계).